

# CB3 Data Story — Architecture Deep Dive

---

This document explains how the whole application is put together: the build-time data pipeline, the runtime React/MapLibre app, and how the two halves meet. It is a deeper companion to the top-level `README.md`. Where the README gives you the map, this gives you the terrain.

It is written for someone comfortable reading code but new to this codebase (and newish to web development). Concepts are explained the first time they appear.

The hover-tooltip subsystem has its own dedicated walkthrough, `docs/tooltips-explained.md`. This file references it but does not repeat it. (The tooltip *code* now lives in `src/map/tooltip.ts`; see §7.)

---

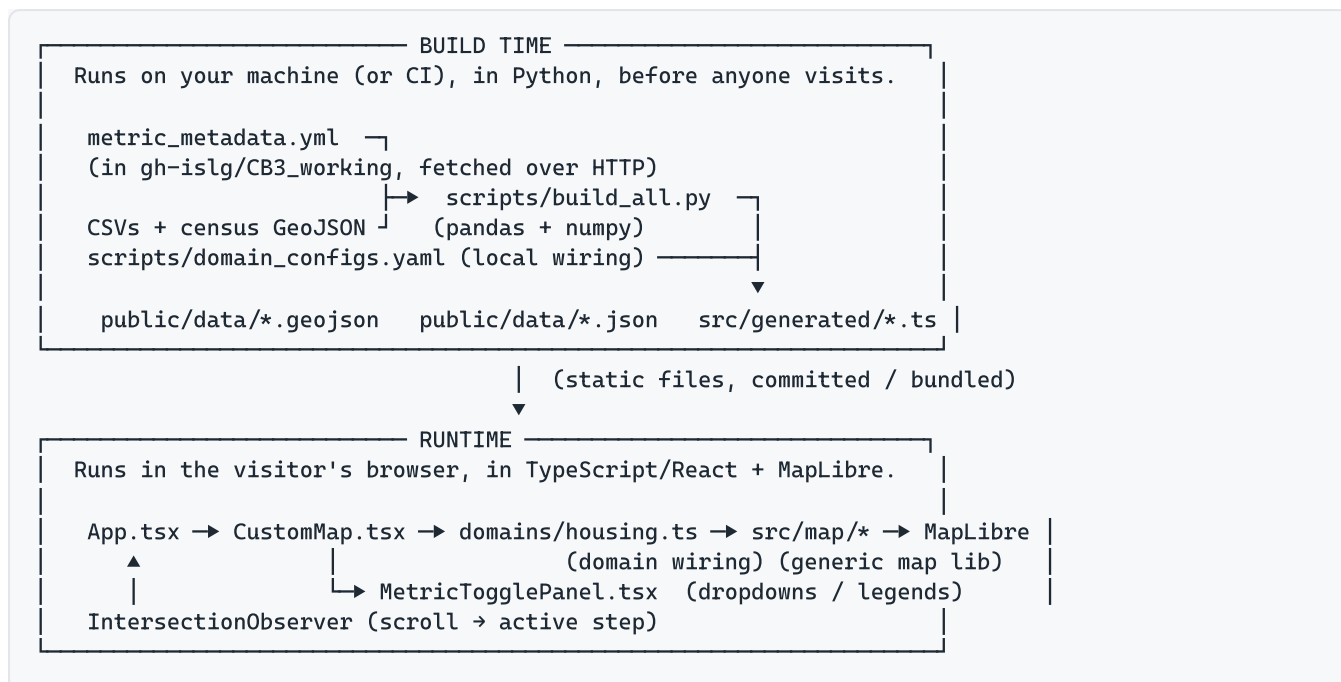
## 1. The one-paragraph mental model

The app is a **scrollytelling** data story: a fixed title on the left, then a tall column of text cards, and a **sticky** map on the right that stays in view while you scroll. As each text card scrolls into the middle of the screen, the map reacts — it flies to a new camera position, restyles the community districts, fades in tract outlines or a demographic choropleth, and calls out specific tracts with labelled "spotlight" bubbles. The final step turns the map into an **interactive** explorer with two dropdown panels and an info card. All of the map's *data* is prepared ahead of time by a Python script and shipped as static files; the browser only ever *reads* those files, never computes them. This split — **heavy work at build time, light work at runtime** — is the single most important architectural idea in the project.

---

## 2. Two worlds: build time vs. runtime

Almost every question about this codebase becomes easy once you know which of two "worlds" a piece of code lives in.



**Why the split matters.** Computing quintile color breaks, projecting bubble sizes, formatting display strings, and merging half a dozen CSVs is slow and needs pandas/numpy — tools that don't exist in a browser. Doing it once at build time means the browser downloads a handful of ready-to-draw files and does almost no math. It also means the *single source of truth* for what a metric means lives in data files, not scattered through the UI code.

### 3. Build time: the data pipeline

#### 3.1 The inputs

| Input                                    | Where it lives                                                                                                             | What it provides                                                                                                                             |
|------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <code>metric_metadata.yml</code>         | separate repo <code>gh-islg/CB3_working</code><br>(fetched over HTTP, cached to <code>scripts/metric_metadata.yml</code> ) | each metric's <code>short_label</code> , <code>palette</code> , <code>unit</code> , <code>domain</code> , <code>positive_values_only</code>  |
| Census tract GeoJSON + CSVs              | <code>public/data/</code>                                                                                                  | tract polygon shapes; per-tract and per-building metric values                                                                               |
| <code>scripts/domain_configs.yaml</code> | this repo                                                                                                                  | wiring: which CSV feeds which metric, join keys, centroid columns, pie/group/point overrides, plus <code>metric_info</code> (info-card copy) |

The YAML is the **source of truth for metric semantics**; `domain_configs.yaml` is the source of truth for **where the data is, how to shape it, and reader-facing copy this repo owns**. Keeping them separate lets the same metric definitions be reused by other projects (the Folium notebooks in `CB3_working`) while this repo owns only its own plumbing.

### 3.2 `build_all.py` — stage by stage

The script runs once per **domain** (currently just `housing_and_affordability`). For each domain it produces five outputs. Here is the flow of `main()` :

```
fetch_metric_metadata()      # download YAML (fallback: cached copy)
├
└ for each domain in domain_configs.yaml:
    build_metric_specs()      # YAML + domain_configs → ordered metric "specs"
    tag_each_spec's_placement # "tract" (centroid) vs "building" (real address)
    merge_choropleth()       # tract polygons + demographic values + *_display → *_tracts.geojson
    build_bubbles()          # point features + precomputed radii + *_display → *_bubbles.geojson
    build_choropleth_legend() # quintile breaks + color bins
    (bubble legend from build_bubbles)
    write_metric_registry()   # specs → auto-generated TypeScript → src/generated/*.ts
                                ↳ *_legend.json
```

(The NTA overlay file, `cb3_ntas.geojson`, is a one-off derived asset — see §3.9.)

### 3.3 Metric specs, and the three "layer types"

`build_metric_specs()` walks the YAML and turns each metric into a **spec** dict. The key field it computes is `layer_type`, decided by which override block in `domain_configs.yaml` (if any) names the metric:

- **bubble** — the default. One plain circle, sized by the metric's value. (Severe rent burden, crowded housing.)
- **pie** — under `pie_overrides`. A donut split into colored wedges; the circle is sized by the **sum** of its wedge values. (New affordable housing by AMI band; HPD violations by Class B/C.)
- **grouped** — under `group_overrides`. Several related series drawn as separate same-shape circles in different colors, toggled as one unit. (Subsidized units by expiration window.)

For pies and groups, the spec carries an ordered `layers` list — the wedge or member definitions `{property, color, label}`, plus an optional `shortLabel` (a shorter wedge label used only in the tooltip; the legend keeps the full `label`). Plain bubbles get `layers: []`.

A subtle step in `main()`: the **sub-components** of a pie or group (individual wedges / non-parent members) are *excluded* from the top-level metric list, so they don't appear as their own dropdown options. Only the "parent" metric survives as a selectable metric.

### 3.4 `placement` : the flag that drives the tooltips

After building specs, `main()` tags each with `placement`, either `"tract"` or `"building"`: a bubble is **building**-placed if it draws from a point file with real lot coordinates; **tract**-placed if it sits at a tract centroid. This one flag decides, at runtime, whether a bubble gets its own hover card or shares the whole-tract card (see `docs/tooltips-explained.md`).

### 3.5 `merge_choropleth()` — the demographic backdrop

Reads the census tract GeoJSON, keeps only CB3 tracts ( `cdta2020 == "MN03"` ), and merges each demographic metric's value onto every tract. Crucially, it writes **two** fields per metric: the raw number `<property>` (the choropleth's `step` color expression needs a *number* to compare against the quintile breaks) **and** a pre-formatted `<property>_display` string (e.g. `"$39,961"` , `"29.1%"` ) for the tooltip to print verbatim.

**Why both?** The formatting used to happen in JavaScript. It now happens once in Python so the tooltip and the legend can never drift, and so browsers with different locales all read the same numbers. The raw value stays because the GPU color expression can't compare against a formatted string.

Safety checks: `n_expected: 31` makes the script **raise** if it doesn't end up with exactly 31 CB3 tracts; unmatched tracts are kept with `None` values and warned about. Output: `cb3_housing_tracts.geojson` .

### 3.6 `build_bubbles()` — points and precomputed radii

Emits **point** features (one dot per tract centroid or per building), each carrying a **precomputed radius** so the browser never sizes a bubble itself. The radius formula ( `make_radius_fn` ) is a **square-root scale**:

```
radius = MIN_RADIUS + (MAX_RADIUS - MIN_RADIUS) * (value / max_value) ** 0.5
```

with `MIN_RADIUS = 5` , `MAX_RADIUS = 22` pixels. Square-root (not linear) scaling makes a circle's **area** — what the eye judges — proportional to value.

Each feature also carries a `<property>_display` string. Type-specific notes: tract-centroid bubbles merge *all* tract-level metrics onto one point (so one point serves several selectable layers, filtered at runtime by `["has", "<m>_radius"]` ); pies are sized by the **sum** of their wedges and carry each wedge's raw count; grouped members share **one** radius scale so equal sizes mean equal counts across colors. Output: `cb3_housing_bubbles.geojson` .

### 3.7 `build_choropleth_legend()` — quintile color bins

For each demographic metric, `compute_quintile_breaks()` cuts the tract values at the 0/20/40/60/80/100th percentiles. Those six breakpoints define five color bins from the metric's light→dark palette. The legend JSON stores both the raw `breaks` (numbers the map uses) and the human `bins` ( `{color, label}` rows the panel renders). Output: `cb3_housing_legend.json` ( `demographics` + `housing` sections).

### 3.8 write\_metric\_registry() — generated TypeScript

The specs are serialized to `src/generated/housingMetrics.ts` as two `as const` arrays: `HOUSING_METRICS` (with `layerType`, `placement`, `layers` incl. optional `shortLabel`, and — where `domain_configs.yaml` 's `metric_info` provides it — `description` + `source` for the info card) and `DEMOGRAPHIC_METRICS` (with `palette`, `positiveOnly`). The header says **do not edit by hand**. `as const` lets the app derive precise union types (e.g. `HousingMetricId`) from the data. This file is the **contract** between the two worlds.

### 3.9 The NTA overlay ( `cb3_ntas.geojson` )

A one-off derived asset (not part of the per-run pipeline): the census tracts are dissolved by NTA name (using `geopandas`) into three neighborhood polygons — East Village, Lower East Side, Chinatown–Two Bridges — carrying an `nta_name` property. Used to bold-outline one neighborhood on the crowding step (§7).

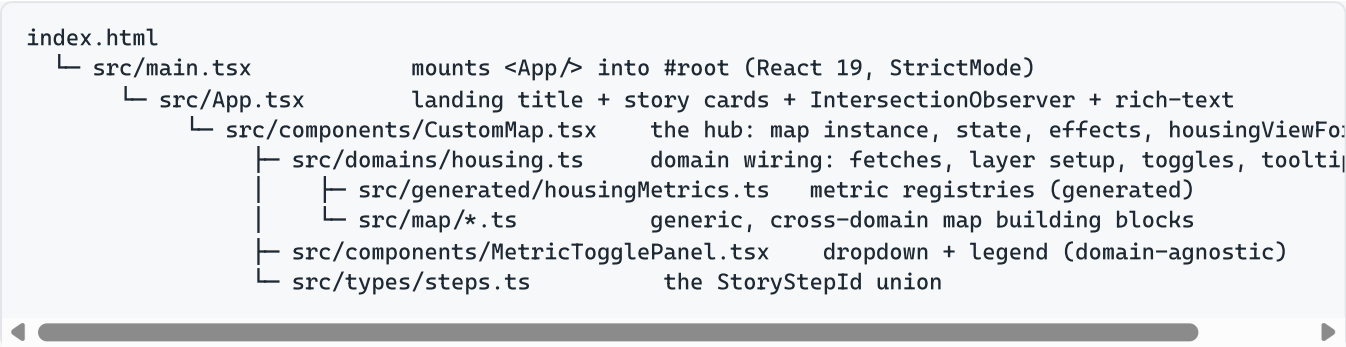
## 4. Runtime: the five static outputs and who reads them

| File                                              | Read by                                          | Purpose                                                                                                             |
|---------------------------------------------------|--------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <code>cb3_housing_tracts.geojson</code>           | <code>setupHousingLayers</code>                  | polygon source for the choropleth, the invisible tract hover fill, the fading tract outline, and spotlight outlines |
| <code>cb3_housing_bubbles.geojson</code>          | <code>setupHousingLayers</code>                  | point source for every bubble/pie/group layer + the tract-card centroid lookup + spotlight markers                  |
| <code>cb3_housing_legend.json</code>              | <code>setupHousingLayers</code><br>→ React state | color bins and bubble-size rows for the legend panels                                                               |
| <code>cb3_ntas.geojson</code>                     | <code>setupHousingLayers</code>                  | neighborhood polygons for the NTA highlight                                                                         |
| <code>src/generated/housingMetrics.ts</code>      | imported directly                                | metric lists driving dropdowns, layers, tooltips, and info cards                                                    |
| <code>Community_Districts_20260615.geojson</code> | <code>CustomMap</code>                           | the 59 community-district polygons (the base-map framing)                                                           |

The `GeoJSON/JSON` files are fetched over HTTP at map-load; the generated `.ts` is bundled at build time because it's source code.

**Deploy gotcha.** `.gitignore` blanket-ignores `public/data/*` and re-allows the shipped files by an explicit allowlist ( `!public/data/cb3_ntas.geojson` , ...). A file that isn't allowlisted (or is misspelled in the allowlist) works locally — Vite serves it from disk — but is never committed, so Azure never deploys it. If a data layer shows locally but not in production, check the allowlist first.

## 5. Runtime: the module map



### 5.1 Each file's single responsibility

| File                             | Job — and what it deliberately does <i>not</i> do                                                                                                                       |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| types/steps.ts                   | Names the story steps as a string union. Nothing else.                                                                                                                  |
| generated/housingMetrics.ts      | Lists the metrics. Auto-generated; never hand-edited.                                                                                                                   |
| map/*.ts                         | <b>Generic</b> MapLibre building blocks — no housing knowledge (§6).                                                                                                    |
| domains/housing.ts               | <b>Housing wiring</b> : which files to fetch, which layers to build, the housing metric loops, and the config it hands the generic modules. The domain's public facade. |
| components/MetricTogglePanel.tsx | Renders a dropdown + legend from props. Never touches the map; doesn't know it's "housing".                                                                             |
| components/CustomMap.tsx         | The hub. Holds state, runs effects, maps each step to a view, wires map ↔ UI.                                                                                           |
| App.tsx                          | Landing title, scroll detection, text cards, tiny rich-text parser. Never touches the map directly.                                                                     |

Two themes: **only `map/*` and `housing.ts` import MapLibre** (React calls their exported functions, never touches layers directly); and **`housing.ts` is the facade** — `CustomMap` imports everything from `housing.ts` , which re-exports the generic pieces ( `GRAY` , `showSpotlight` , ...) so callers have one import surface.

## 6. Runtime: the `src/map/` shared layer

This is the newest structural idea. As the housing module grew, the parts that were *generic MapLibre mechanism* (as opposed to *housing data/wiring*) were pulled into `src/map/`, so a future domain can reuse them. Each module is dependency-light and takes explicit config — no domain knowledge baked in.

| Module                      | Exports                                                                                                                                     | What it is                                                                  |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| <code>colors.ts</code>      | <code>GRAY</code> , <code>BUBBLE_COLOR</code> , <code>BUBBLE_OUTLINE_COLOR</code>                                                           | shared palette constants                                                    |
| <code>markers.ts</code>     | <code>buildPieElement</code> , <code>buildPillLabel</code> ,<br><code>buildMetricBubble</code>                                              | DOM-element builders for map markers (donuts, pills, orange metric bubbles) |
| <code>expressions.ts</code> | <code>buildColorExpression</code>                                                                                                           | builds the MapLibre <code>step</code> choropleth color expression           |
| <code>tractLayers.ts</code> | <code>addTractHoverLayer</code> , <code>addTractOutlineLayer</code> ,<br><code>addFadingTractOutline</code> , <code>fadeTractOutline</code> | tract fill/outline layer builders + the opacity-fade toggle                 |
| <code>tooltip.ts</code>     | <code>installMapTooltips</code> , <code>TooltipMetric</code> ,<br><code>TooltipConfig</code>                                                | the reused popup, the card builders, and the mousemove handler              |
| <code>spotlight.ts</code>   | <code>registerSpotlight</code> , <code>resetSpotlights</code> ,<br><code>showSpotlight</code>                                               | a registry of named marker+outline groups and the on/off toggle             |
| <code>highlight.ts</code>   | <code>addFeatureHighlightLayer</code> ,<br><code>showFeatureHighlight</code>                                                                | highlight one feature of an overlay source by a property value              |

The dividing line: **mechanism goes in `src/map/`; data/wiring stays in `housing.ts`**. For example, `spotlight.ts` owns the registry and the show/hide logic, but *which tracts* each spotlight highlights, and the marker-building loops, live in `housing.ts`. The one assumption `src/map/` bakes in is the **shared pipeline field contract** (`<property>_display`, `tract_label`, `nta_name`, `placement`, `GEOID`) that `build_all.py` produces for every domain.

## 7. Runtime: the MapLibre layer model

The visual is built from **sources** (raw GeoJSON) and **layers** (drawing rules over a source). One source can back many layers. The project leans on a favorite trick: **invisible-but-queryable** layers.

### 7.1 What `setupHousingLayers()` registers

From `cb3_housing_tracts.geojson` (source `housing-tracts`): - **tract-hover** — a fully transparent fill over every tract; invisible but a real queryable layer, the universal "is the mouse over a tract?" target for tooltips (`addTractHoverLayer`). - **one fill + one outline per demographic metric** — the

choropleth backdrop, each pair hidden until selected. The fill color comes from

`buildColorExpression()` . - **demographic-none-outline** — a thicker orange tract boundary that *fades* in via `line-opacity` (`addFadingTractOutline`), shown for the "no demographic background" choice and to ease the 31 tract boundaries in on `zoom-cb3-tracts` . - **spotlight outline layers** — one thick orange outline per spotlight that has one (`addTractOutlineLayer`), filtered to that spotlight's tracts, hidden until shown.

From `cb3_ntas.geojson` (source `cb3-ntas`): the **NTA highlight** line layer (`addFeatureHighlightLayer`), hidden until narrowed to one neighborhood.

From `cb3_housing_bubbles.geojson` (source `housing-bubbles`): **one circle layer per bubble/grouped metric**, filtered by `["has", "<property>_radius"]` and sized by `["get", "<property>_radius"]` . Plus **spotlight markers** and **pie donuts** (DOM markers — next).

## 7.2 `buildColorExpression()` — data-driven choropleth color

The choropleth doesn't recolor tracts in JavaScript; it hands MapLibre a **step expression** the GPU evaluates per feature:

```
["step", ["to-number", ["get", property], 0], color0, break1, color1, break2, color2, ...]
```

Interior quintile breaks (p20–p80) become the boundaries. `positiveOnly` metrics are wrapped in a `case` that paints zero/absent tracts gray.

## 7.3 Pies: DOM markers + an invisible hover circle

A conic-gradient wedge fill isn't a MapLibre paint property, so pies are **DOM markers** — HTML `<div>`s (built by `buildPieElement`) with CSS `conic-gradient` wedges. Two consequences: they're made **click-through** (`pointerEvents = "none"`) so hover falls through to the map, and — for building-placed pies — an **invisible circle layer** sits beneath them as the queryable hover target. Because markers aren't in MapLibre's layer stack, "showing/hiding" a pie means attaching/detaching the markers.

## 7.4 Toggling & fading

- `showHousingMetric(map, id | null)` — radio-group over the metrics. Pies attach/detach markers (+ toggle the hover circle); grouped members toggle together; **plain bubbles fade** via `circle-opacity` transitions (so severe-rent bubbles ease in when you scroll `housing-intro` → `housing-rent-burden`). `null` hides everything.
- `showDemographicLayer(map, id | null)` — shows one choropleth pair, or fades the orange tract outline for the "no background" sentinel, or hides all.

Several **animated transitions** make the scroll feel smooth: the district `fill-opacity` (800 ms, so CB3's fill dissolves on `zoom-cb3-tracts`), the tract outline `line-opacity` (600 ms), and the bubble `circle-opacity` (500 ms).

## 7.5 Spotlights

A **spotlight** is a named set of DOM markers — orange metric bubbles (the value in white inside, via `buildMetricBubble`) and/or income "pill" labels (`buildPillLabel`) — plus optionally one thick tract outline, all toggled together to call out specific tracts on one step. `housing.ts` builds them and `registerSpotlight`s them by id (`fade-others-income`, `rent-burden`, `crowding`); `showSpotlight` toggles one on/off. `CustomMap` shows the active one and hides the rest each step.

## 7.6 NTA highlight

`showNtaHighlight(map, ntaName | null)` (a thin wrapper over the generic `showFeatureHighlight`) narrows the NTA line layer's filter to one `nta_name` and shows it — used to bold-outline Chinatown–Two Bridges on the crowding step.

## 7.7 Tooltips (see `docs/tooltips-explained.md`)

The hover system now lives in `src/map/tooltip.ts` as `installMapTooltips(map, config)`: one reused popup, the tract/building card builders (which read only the shared pipeline fields), and a mousemove handler that queries building layers first, then the tract-hover fill. `housing.ts`'s `installHousingTooltips` is a thin wrapper supplying the housing `TooltipConfig` (metric getters, which layers to query, the centroid lookup).

---

## 8. Runtime: `housingViewFor` — the per-step view resolver

---

The heart of the runtime is one pure function in `CustomMap.tsx`:

```
housingViewFor(step, userMetric, userDemographic) → {  
  metric, demographic, panels, nta, spotlight  
}
```

It maps each `StoryStepId` to a **declarative description of what the map should show**: which bubble metric, which choropleth backdrop, whether the interactive dropdown panels appear, which NTA to highlight, and which spotlight to reveal. Reading it is the fastest way to understand the whole narrative:

| Step                | metric (bubbles)     | demographic (fill)         | panels     | nta                   | spotlight               |
|---------------------|----------------------|----------------------------|------------|-----------------------|-------------------------|
| fade-others         | —                    | median income              | no         | —                     | income (22.02 vs 26.01) |
| zoom-cb3-tracts     | —                    | <i>none</i> (bare outline) | no         | —                     | —                       |
| housing-intro       | —                    | median income              | no         | —                     | —                       |
| housing-rent-burden | severe rent burden   | median income              | no         | —                     | rent burden (8 vs 42)   |
| housing-crowding    | crowded housing      | %LEP                       | no         | Chinatown–Two Bridges | crowding                |
| housing-map         | <i>user's choice</i> | <i>user's choice</i>       | <b>yes</b> | —                     | —                       |
| (anything else)     | —                    | —                          | no         | —                     | —                       |

Only `housing-map` reads the user's dropdown selections; every narrative step **pins** a fixed metric + demographic and keeps the panels hidden. Because it's one function, "what does step X show?" and "add a new step" are both one edit here.

## 9. Runtime: state, refs, and the effect chain

`CustomMap.tsx` holds a little React **state**, a matching set of **refs**, and **three** `useEffect` blocks.

### 9.1 State it owns

```
const [activeMetric, setActiveMetric] = useState(DEFAULT_HOUSING_METRIC_ID); // "subsidized housing"
const [activeDemographic, setActiveDemographic] = useState(DEFAULT_DEMOGRAPHIC_METRIC_ID); // "poverty rate"
const [housingLegend, ...] / [demographicLegend, ...] // legend data, set once at load
```

`activeMetric` / `activeDemographic` are the **user's two independent dropdown selections** on `housing-map` (they default to subsidized housing + poverty rate). `activeStep` is *not* owned here — it arrives as a prop from `App`.

### 9.2 Why refs shadow the state

The map is created **once** inside `map.on("load", async () => ...)`. That closure captures values as they were at creation, but the housing fetch is `await`-ed — by the time it resolves, the user may have scrolled or changed a dropdown. Reading the captured value would be **stale**. So each value is mirrored into a **ref** (`someRef.current`) that survives re-renders; the effects write the latest value, and the `async`

closure reads `.current`. There are also `shownMetricRef` / `shownDemographicRef` holding the *effective* (possibly preset) metric/demographic, which the tooltip getters read so a hover card matches whatever is actually on screen — not the dropdown state.

### 9.3 The three effects

```
Effect 1  [] once    create the map; on load add district layers, await
                    setupHousingLayers(), install tooltips, then re-apply the
                    current step from the refs (§8's view + all the show* calls).

Effect 2  [activeStep]          applyMapState(map, step) - camera + districts
Effect 3  [activeStep, activeMetric, activeDemographic]
          const view = housingViewFor(...);
          showHousingMetric / showDemographicLayer / showNtaHighlight
          + toggle each spotlight (active one on, rest off)
```

Effect 3 replaced the old separate metric/demographic effects: since one function (`housingViewFor`) now derives everything, one effect applies it all. Guard clauses (`if (!map || !mapReadyRef.current) return;`) make the effects no-ops until the map exists.

### 9.4 `applyMapState()` — the base-map director

A plain function that, for each step, sets the camera (`easeTo` / `fitBounds`) and the community-district paint. The ladder:

- `citywide` — zoomed out (~9.8), all 59 districts faint, CB3 orange. (*This is the landing default — see §11 — not a scroll card.*)
- `highlight-cb3` — zoom 12, CB3 fill strengthened.
- `fade-others` — zoom 14, CB3 fill → transparent (income choropleth shows), others a heavy white mask. Mask/outline values match the housing block so the transition into `housing-intro` has no visible jump.
- `zoom-cb3-tracts` — `fitBounds` to CB3, CB3 fill fades to 0 (tract outlines pop).
- **any `housing-*` step** (`isHousingStep`) — stay at zoom 14, CB3 fill transparent so the housing choropleth shows, others white-masked.

`applyMapState` handles **only** the base map (districts + camera); the housing layers are shown/hidden separately by Effect 3, keeping choreography and metric-toggling from tangling.

---

## 10. Runtime: the UI panels & info card

`MetricTogglePanel` is a **presentational** component: given `metrics`, `legendData`, `activeMetric`, and `onMetricChange`, it renders a `<select>` plus a legend. Two behaviors via one `interactive` prop:

- **Interactive** (`housing-map`): the `<select>` dropdown; changing it calls `setActiveMetric` / `setActiveDemographic` → Effect 3 → toggles layers.

- **Read-only** (narrative preset steps): the same legend, but the dropdown is replaced by a static heading naming the fixed metric — so the reader sees the key without controls.

It's deliberately domain-blind (never imports `housing.ts`, never touches the map). Two panels stack in opposite corners via `panelClassName`. Alongside them, on `housing-map` only, a gray **info card** (bottom-left) shows the active metric's `description` + `source` (from `metric_info`), for any metric that has them.

## 11. The story: landing title, steps, and rich text

`App.tsx` owns the narrative column. The `storySteps` array is the ordered list of scroll cards; each has an optional `eyebrow`, `title`, `body`, and `footnote`.

- **The landing title** ("MCB3 commissioned this analysis...") is a fixed `.intro` block, **not** a scroll card. `citywide` remains a valid `StoryStepId` used only as the map's initial `activeStep` default (giving the wide citywide camera behind the title); with no `<article>`, the scroll observer can never re-select it.
- **The scroll cards** run: `highlight-cb3` → `zoom-cb3-tracts` → `fade-others` → `housing-intro` → `housing-rent-burden` → `housing-crowding` → `housing-map`.
- **Rich text.** Step copy is plain strings run through a tiny parser (`formatInline` / `formatLines` / `formatBlock`, defined at the top of `App.tsx`): ***\*\*bold\*\****, ***\*italic\****, `\n` line breaks, and `\n\n` paragraph breaks (body only). It's intentionally minimal — not a markdown library.

## 12. End-to-end flows

## 12.1 A scroll changes the step

User scrolls

- IntersectionObserver in App.tsx picks the most-visible [data-step-id] card
- setActiveStep("housing-rent-burden")      React state in App
- App passes activeStep to <CustomMap>
- Effect 2 → applyMapState()                  camera + district styling
- Effect 3 → housingViewFor(step) →        showHousingMetric / showDemographicLayer  
                                                   / showNtaHighlight / showSpotlight
- CustomMap re-renders → read-only legend panels appear for the step's fixed layers

The observer's thresholds + `rootMargin` bias toward the **center** of the viewport, so a step activates when it's centrally in view.

## 12.2 A dropdown changes the metric (housing-map only)

```
User picks "Crowded housing"
→ MetricTogglePanel: onMetricChange("crowded-households-pct")
→ setActiveMetric(...) in CustomMap
→ Effect 3 → housingViewFor("housing-map", ...) → showHousingMetric(...)
→ crowded bubbles fade in; others fade out; legend + info card update
```

No data is refetched — everything was loaded once at map-load and is just toggled.

## 13. Rendering & deployment

| Layer         | Technology                                                                                                |
|---------------|-----------------------------------------------------------------------------------------------------------|
| Hosting       | Azure Static Web Apps (workflow under <code>.github/workflows/</code> )                                   |
| Map renderer  | MapTiler SDK wrapping MapLibre GL JS                                                                      |
| Frontend      | React 19 + TypeScript, bundled by Vite                                                                    |
| Data pipeline | Python ( <code>build_all.py</code> ) with pandas + numpy (+ geopandas for <code>cb3_ntas.geojson</code> ) |
| Map data      | MapTiler vector tiles + local static GeoJSON in <code>public/data/</code>                                 |

- **Local dev:** `npm run dev`. The MapTiler key is read from `.env` as `VITE_MAPTILER_API_KEY` (Vite only exposes `VITE_`-prefixed vars). `.env` is git-ignored.
- **Production build:** `npm run build` runs `tsc` then `vite build` → `dist/`. `public/` is copied to the site root, so `public/data/foo.geojson` serves at `/data/foo.geojson` — exactly what `setupHousingLayers` fetches.
- **Deploy:** pushing triggers the Azure Static Web Apps Action, which builds and publishes `dist/`. **Remember the data allowlist** (\$4) — an un-allowlisted data file deploys as a 404.

The data pipeline is **not** part of `npm run build`. Run `python scripts/build_all.py` by hand when metrics/data/copy change, and commit the outputs.

## 14. Extending the app

### 14.1 Add / adjust a housing metric or its copy

1. Metric definition (label, palette, unit): `metric_metadata.yaml` in `CB3_working`.
2. Shaping (pie/group/building) and reader copy ( `metric_info` , `short_label` ): `scripts/domain_configs.yaml`.
3. Run `python scripts/build_all.py` and commit the regenerated files.

## 14.2 Add a narrative step or spotlight

1. Add the step id to `src/types/steps.ts` ; add a card to `storySteps` in `App.tsx` .
2. Add a `case` to `housingViewFor` (metric/demographic/panels/nta/spotlight).
3. Camera/base-map look: add/extend a branch in `applyMapState` .
4. A new spotlight: add its id + `SPOTLIGHT_TRACTS` entry and a `registerSpotlight` call in `housing.ts` (and add it to `ALL_SPOTLIGHTS` in `CustomMap` ).

## 14.3 Add a whole new domain (e.g. "environment")

1. Add its metrics to `metric_metadata.yml` ; add a domain entry to `domain_configs.yml` (commented template at the bottom); run the pipeline.
2. Write an `environment.ts` domain module that reuses the `src/map/*` generics (choropleth, bubbles, tooltips via `installMapTooltips` , spotlights, ...) and supplies its own data/config — the same shape `housing.ts` uses.
3. Add its steps to `steps.ts` / `App.tsx` / `housingViewFor` (or a domain-specific equivalent) and `applyMapState` .

Because the metric registry is generated and the map mechanics are generic, steps 1 are data work and the code additions are domain wiring, not new mechanism.

---

## 15. The whole thing in one breath

---

A Python script turns a YAML metric catalog, local CSVs, and census shapes into five static files: tract polygons (raw values + pre-formatted display strings), bubble points (precomputed sizes), a legend, dissolved neighborhood polygons, and a generated TypeScript metric registry. In the browser, React shows a scrolling story whose active step is detected by an `IntersectionObserver`; a sticky MapLibre map reacts to each step. One pure function, `housingViewFor` , maps each step to a view — which bubble metric, which choropleth, whether the dropdowns show, which neighborhood to outline, which tracts to spotlight — and one effect applies it, with animated fades. The generic map mechanics live in `src/map/` ; `housing.ts` supplies the domain's data and wiring; refs keep a one-time async load callback reading fresh values. Nothing heavy happens at runtime; the browser only reads what the build already computed.